Informe de seguridad

Pentest controlado (Demo SaaS + API + n8n) – Enfoque externo tipo bot

Cliente / Equipo	[redacted]
Aplicación objetivo	[redacted]
Tipo de prueba	Black-box externo + usuario estándar (no admin)
Perímetro observado	Cliente Cloudflare Firewall proveedor VPS VPS
Componentes declarados	docker-compose full-stack (front/back/db) + docker-compose n8n (webhooks expuestos)
Backup	No disponible (se espera durante la semana)
Fecha	22/01/2026
Autor	Javi
Versión	v1.1 (plantilla)

Nota: plantilla operativa. Completa los campos, añade evidencias y registra cambios durante la revisión.

Índice

1. Resumen ejecutivo
2. Alcance, exclusiones y reglas de compromiso (RoE)
3. Arquitectura observada y supuestos
4. Metodología, herramientas y flujo de trabajo (roadmap)
5. Evidencias y trazabilidad
6. Hallazgos (tabla) y priorización
7. Recomendaciones (quick wins) y plan de mitigación
8. Anexos (endpoints, capturas, exports).

1. Resumen ejecutivo

Objetivo: identificar vulnerabilidades básicas desde la perspectiva de un atacante externo (bot/script genérico), con acceso de usuario estándar proporcionado por el equipo. Prueba no destructiva y sin afectar al servicio.

Situación actual (inputs confirmados):

Hay Cloudflare delante (cabecera 'server: cloudflare').
Perímetro declarado: Cliente Cloudflare firewall proveedor VPS VPS.
Stack: docker-compose full-stack (front/back/db) con API expuesta; docker-compose n8n con webhooks expuestos.
No hay backup todavía. Por tanto, Gate 0 es HARD: no automatización activa sin OK explícito.

Resultados clave (rellenar al finalizar):

Hallazgos críticos/altos detectados:
Quick wins recomendados:
Riesgos residuales y siguientes pasos:

2. Alcance, exclusiones y reglas de compromiso (RoE)

2.1 Alcance

Dominio objetivo: dapp.aishophelper.ai (y subdominios relacionados detectados).
Modalidad: Black-box externo con usuario estándar (no admin).
Enfoque: "bot-like" + validación manual de lógica (IDOR/BOLA) + revisión pasiva de webhooks n8n.

2.2 Exclusiones

No DoS, no fuerza bruta, no fuzzing masivo sin límites.
No intentos de bypass/evadir Cloudflare/WAF.
No extracción de datos ajenos. Si se detecta acceso indebido, se documenta y se para.
Infraestructura interna (SSH/BD) fuera de alcance salvo autorización explícita posterior.

2.3 Gate -1 y Gate 0 (HARD)

Gate -1 (operativo): acordar límites de ritmo, condiciones de parada y ventana de pruebas. Gate 0 (HARD): con ausencia de backup, NO se ejecutan herramientas activas/automatizadas (ZAP Active Scan, Nikto, SQLMap, XSStrike, crawlers agresivos) hasta completar Fase 5 y contar con OK explícito.

2.4 Condiciones de parada (Stop conditions)

Degradación visible del servicio o aparición de 5xx repetidos.
Indicios de ejecución de acciones reales (emails, cambios persistentes, ejecuciones n8n).
Confirmación de acceso a datos ajenos (IDOR/BOLA).

3. Arquitectura observada y supuestos

3.1 Diagrama (declarado por el equipo)

Cliente

Cloudflare

Firewall proveedor VPS

VPS
- docker-compose full-stack (front, back y db) – API expuesta
- docker-compose n8n – webhooks expuestos

3.2 Supuestos y preguntas abiertas (a completar)

¿El origen (VPS) solo acepta tráfico 80/443 desde rangos de IP de Cloudflare?
¿Los webhooks de n8n tienen secreto/firma (token/secret header) y rate limiting?
¿Existen entornos separados (demo/staging) o todo vive en el mismo VPS?

4. Metodología, herramientas y flujo de trabajo (roadmap)

4.1 Fases y prioridad

Fase	Objetivo / entregable
Fase 1 — Recon pasivo (desde la calle, Cloudflare-aware)	Tecnologías, TLS, archivos públicos, baseline de headers/cookies.
Fase 2 — Montaje del laboratorio	Navegador limpio FoxyProxy Burp/Caido. CA instalada. Scope estricto. Intercept OFF.
Fase 3 — Mapeo funcional autenticado	Login con usuario estándar. Navegación completa. Mapa de endpoints (API/auth/IDs/errores/terceros).
Fase 4 — Checks manuales bot-like	Headers, cookies, CORS, sesión/logout, leaks, sourcemaps, keys expuestas.
Fase 4.5 — Webhooks n8n expuestos (pasivo)	Identificar webhooks y su comportamiento sin disparar acciones ni payloads.
Fase 5 — Autorización y lógica	IDOR/BOLA controlado, rutas admin (solo ver denegación/redirect).
Gate 0 (HARD)	Automatización activa solo con OK explícito y tras Fase 5. Sin backup: normalmente se pospone.

Fase 6 — Crawling automatizado con límites	katana/hakrawler/ZAP spider: profundidad baja, rate bajo, scope estricto.
Fase 7 — ZAP Active Scan (policy SAFE)	Solo como detector de señales: reglas no agresivas, concurrencia baja, scope estricto.
Fase 8 — Nikto (excepcional)	Solo si hay sospecha de rutas legacy/config; siempre con límites.
Fase 9 — SQLi con SQLMap (modo detección)	Solo si hay indicios, sin dump, sin shell, risk/level bajos.
Fase 10 — XSS con XSStrike (controlado)	Solo en inputs candidatos, sin afectar usuarios reales.
Fase 11 — Reporte	Resumen ejecutivo, tabla de hallazgos, quick wins y plan de mitigación.

4.2 Encadenado (no saltar prerequisitos)

Recon pasivo Proxy listo Login + mapa endpoints Checks manuales IDOR/BOLA
Gate 0 (OK explícito) Automatización SAFE Reporte

4.3 Herramientas y entorno (Kali / Ubuntu)

La ejecución puede hacerse en Kali o Ubuntu. Kali trae más herramientas preinstaladas; Ubuntu es más limpio pero instalas lo mismo. En este proyecto, por Cloudflare + ausencia de backup, se prioriza manual/quirúrgico y se limita la automatización.

Herramientas base usadas:

Navegador con perfil limpio (Chrome/Firefox) + DevTools
FoxyProxy (enrutado navegador proxy)
Burp Suite Community o Caido (proxy/interceptación)
Wappalyzer/BuiltWith (fingerprinting)
Whatweb
curl (pruebas repetibles) + testssl.sh (TLS)
OpenSSL
Shodan / Censys / crt.sh
WPScan (pasivo)

Scripts propios para: Descarga segura de posibles archivos encontrados, Análisis de JS , Enumeracion de rutas (endpoint y Apis)

OSINT de correos y archivos

Herramientas condicionadas al Gate 0 (solo con OK explícito):

Crawler: katana / hakrawler (profundidad y rate limit bajos)
OWASP ZAP (Active Scan con policy SAFE, scope estricto)
Nikto (excepcional, con límites)
SQLMap (detección en modo seguro: --risk=1 --level=1, sin dump/shell)
XSStrike (XSS controlado; solo inputs candidatos)
Hydra carga de payloads y diccionarios

Scanners tipo Nessus/OpenVAS

En este escenario NO se recomiendan al principio: con Cloudflare delante suelen medir Cloudflare (no el origen) y generan ruido. Si se usan, que sea en staging o apuntando al origen con autorización explícita y límites estrictos. (Necesito acceso al servidor o ip real para tal escaneo)

5. Evidencias y trazabilidad

5.1 Evidencias mínimas por fase

Fase	Evidencias
Fase 1	Capturas: Wappalyzer, TLS, robots/sitemap, headers/cookies home+login.
Fase 2	Captura: proxy funcionando + scope.
Fase 3	Export: endpoints top (10–30) + 10 requests representativas.
Fase 4	5–10 evidencias: cookies, CSP/HSTS, CORS, logout, leaks/errores.
Fase 4.5	2–3 endpoints webhook y su respuesta (sin disparar acciones).
Fase 5	2 pruebas IDOR/BOLA con request/response y explicación.

5.2 Registro de acciones (log)

Rellenar durante la ejecución.

1.)Hallazgo: Gestión incorrecta de rutas estándar /well-known

Fecha/hora	Acción	Herramienta	Notas/resultado
22/01/2026/ 15:09	Escaner	whatweb/Curl	No es una vulnerabilidad pero si una grieta importante ya que esta version en concreto a tenido varios CVE
23/01/2026 11:00	Escaner/revision	nevegacion/curl	Actualizacion de version nginx /1.29.4

Descripción:
Al acceder a /well-known/security.txt, el servidor responde con un error **500 Internal Server Error**, exponiendo además la versión exacta del servidor web (nginx/1.22.1).

SHODAN:PRUEBA.

Identificaste la IP públic[redacted]7

Detectaste los subdomini[redacted]

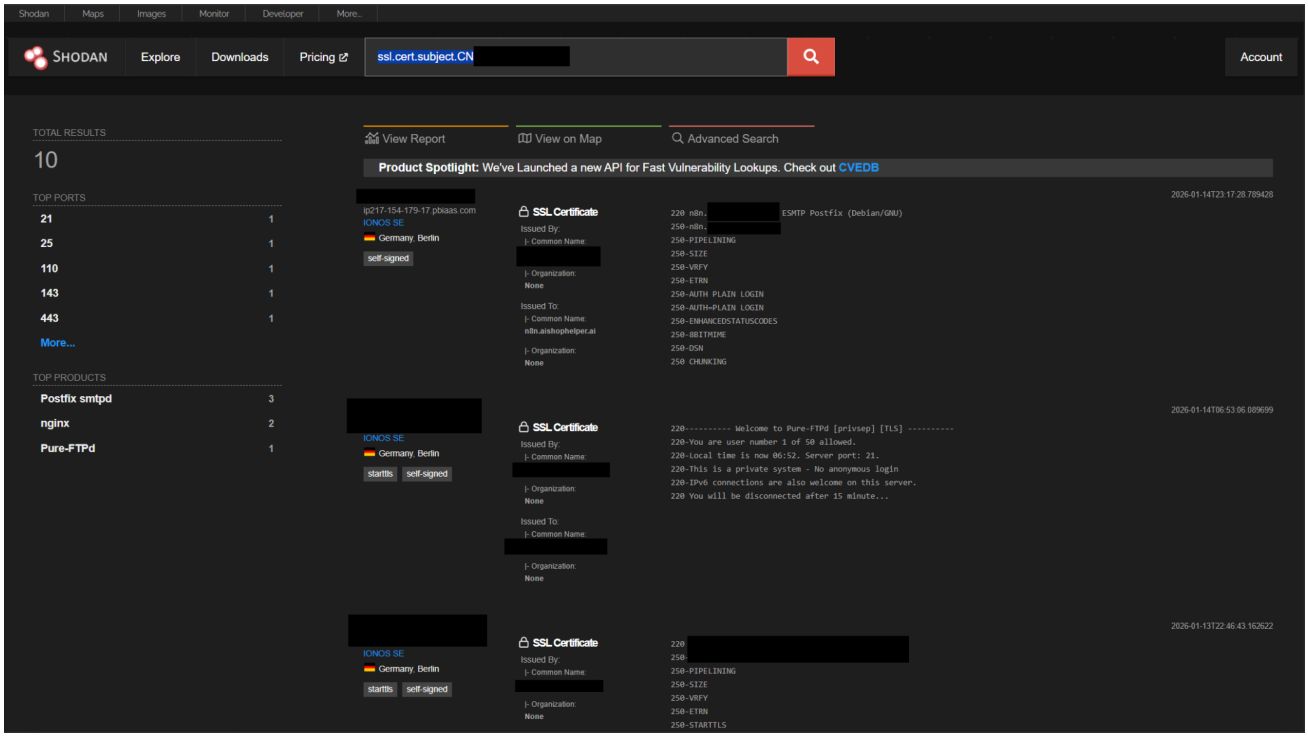
Confirmaste servicios expuestos (antes del filtrado actual)

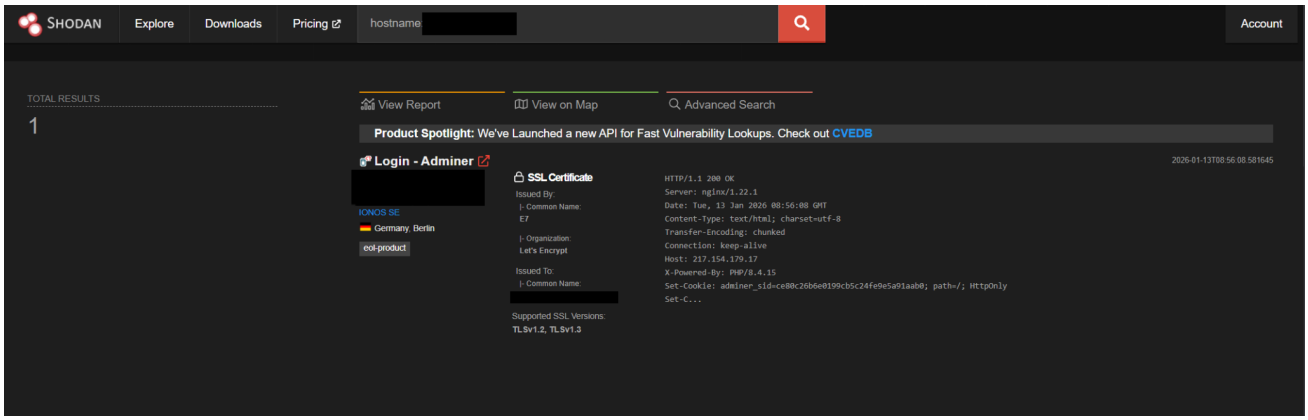
Obtuviste versiones de software (nginx, PHP, Postfix, etc.)

Verificaste que adminer está protegido con Cloudflare Access

Verificaste que los puertos sensibles ahora estfiltrados

Confirmaste que el certificado SSL fue emitido (autofirmado / Let's Encrypt)



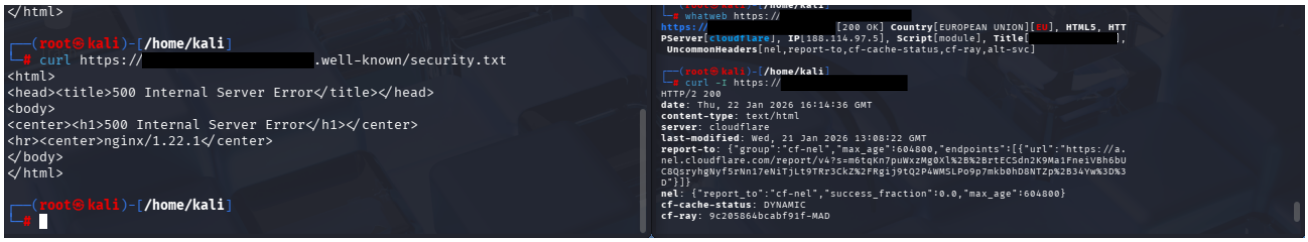


Version de Nginx/1.29.4

Impacto:

Revelación de información del servidor severidad : baja
Gestión incorrecta de errores en rutas estándar severidad : media
Posible indicador de mala configuración del proxy/backend si
Impacto directo Bajo

Evidencia:



VULNERAVILIDADES:

CVE-2023-44487 DoS (HTTP/2) Vulnerable

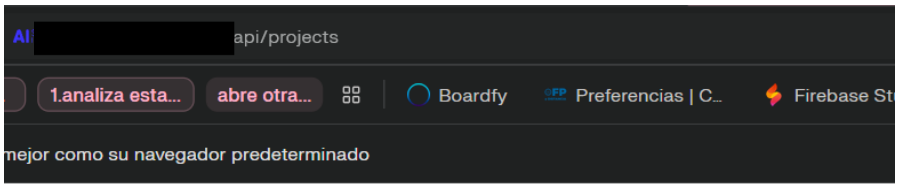
CVE-2022-41741 Memory Corruption Protegido (Fixed)

CVE-2022-41742 Memory Leak Protegido (Fixed)

RECOMENDACION:

Actualizar NGINX 1.28.0 (ramo *stable*), lanzada el **23 de abril de 2025**

Version Nginx/1.29.4 a otro servidor para front-end o por balanceador de carga



404 Not Found

nginx/1.29.4

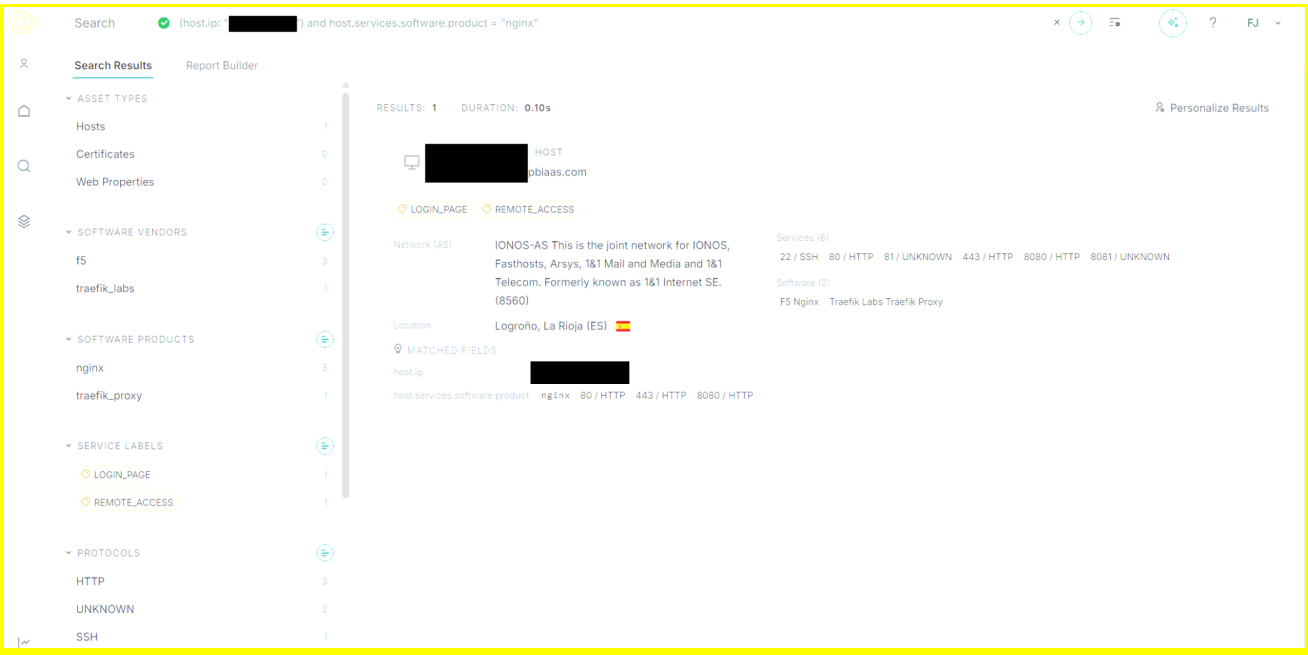
CENSYS:PRUEBA CERTIFICADOS SSL Y PRUEBAS GENERAL.

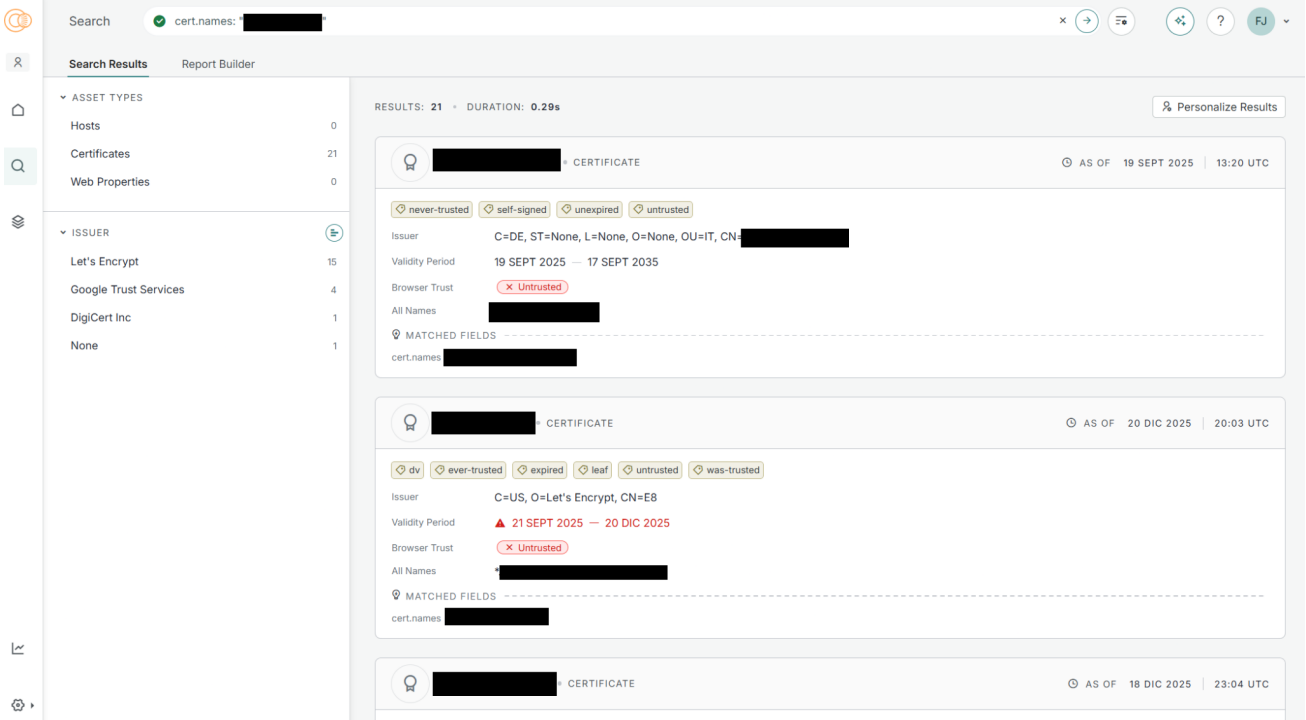
Se identificó la I[redacted]17 como posible servidor origin.

Las pruebas de conexión directa mediante TLS (curl y OpenSSL) muestran cierres abruptos de sesión (unexpected EOF), lo que indica la existencia de controles a nivel de red/TLS.

El servidor origin no acepta conexiones directas por IP ni por Host/SNI manipulados, lo que sugiere **whitelist de direcciones IP (probablemente Cloudflare)** o configuración de **origin-only access**.

No se detectó exposición directa del backend.





Analisis de Cabeceras /sitemal.xml /robots.txt/

Fecha/hora	Acción	Herramienta	Notas/resultado
23/01/2026/ 11:30	Navegacion/Curl	curl	5

Descripción:

Se accedió a la ruta /assets/index-C6MKq1vh.js la cual contiene el archivo principal JavaScript de la aplicación. Este archivo incluye referencias internas a múltiples módulos funcionales de la aplicación, como por ejemplo: reset-password, billing, projects, admin, notifications, entre otros. Estos nombres de archivo permiten inferir posibles rutas internas de la aplicación y funcionalidades sensibles.

Impacto:

La exposición pública de rutas y módulos internos puede facilitar tareas de fingerprinting, recolección de rutas potencialmente sensibles y apoyar ataques como fuerza bruta de rutas, bypass de autenticación o acceso no autorizado a funcionalidades si no están correctamente protegidas por control de acceso en el backend.

Evidencia:

```
┌──(root㉿kali)-[/home/kali]
└─# curl https://[redacted]/sitemap.xml
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>[redacted]</title>
    <link rel="icon" type="image/png" href="/assets/images/favicon-96X96.png"
 />
    <link rel="shortcut icon" href="/assets/images/favicon.ico" />
    <script type="module" crossorigin src="/assets/index-C6MKq1vh.js"></scrip
t>
    <link rel="stylesheet" crossorigin href="/assets/index-XhxsBq1M.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

Fragmento relevante extraido del contenido:/assets/index-C6

1.Archivos .Js analizados

__vite__mapDeps=["assets/reset-password-0O7vXRgw.js",

"https://da[redacted]/assets/general-CH9OwiLY.js ",

verificar qué endpoints y formularios están disponibles dentro de esta sección navegando directamente en la aplicación.

Ruta:/settings

Este componente está asociado al título "titles.settings" y parece estar montado en una ruta de tipo /settings Esto es importante porque:

Normalmente, en esta sección se encuentran funciones críticas como:

Cambiar email
Cambiar contraseña
Configurar MFA
Gestión de sesiones activas
Datos personales

"https://da[redacted]assets/projects-DpiZXxkI.js ",

1. Modificación de roles de usuarios sin validación visible

Hay operaciones críticas como cambiar el rol de un usuario en un proyecto:

```
j.mutate({
  is_administrator: true,
  is_editor: false,
  userProjectId: s.id
```

})

Riesgo potencial:

Si no hay una **verificación de permisos en el backend**, un atacante podría modificar directamente los roles mediante requests manipulados.
Desde el frontend, no hay una validación de que el usuario actual tenga permiso para hacerlo.

Vector potencial: escalación horizontal o vertical de privilegios

2.Falta de confirmación sólida para acciones destructivas.

La acción de borrar un proyecto depende únicamente de una función JS con `confirm(...)`:

```
if(confirm("¿Estás seguro de que quieres borrar este proyecto?")) {

  await B.mutateAsync(s)

}
```

Riesgo potencial:

Si no hay verificación fuerte en backend, podrías explotar esta funcionalidad mediante **CSRF** o scripts automatizados.

"https://da[redacted]assets/change-password-DfOmSuuT.js", HACER PRUEBAS DE

No se observa uso de 2FA, ni verificación adicional (como código por email o token temporal). Esto indica que **cualquier usuario autenticado podría cambiar su contraseña con solo saber la contraseña actual**

Esto por sí solo no es una vulnerabilidad, pero es una oportunidad para:

Brute force sobre `current_password`, si no hay limitación en backend.

Abuso si hay alguna fuga previa de contraseña, por ejemplo por sesión hijack o token comprometido.

---

3. No hay evidencia de rate limiting o detección de intentos fallidos

No se ve implementación de mecanismos para:

Contar cuántos intentos fallidos de cambio de contraseña realiza un usuario.

Bloquear o ralentizar el proceso tras varios errores.

"https://d[redacted]assets/billing-DVh0YBoe.js";: RETORNA A LA WEB. HACER PRUEBAS DE PENTEST

"https://d[redacted] assets/admin-DRrG8g0t.js ",

Aunque no es una vulnerabilidad directa, **este archivo revela una gran cantidad de información sensible** sobre funcionamiento interno del frontend y posibles rutas backend que deberías testear. Es un **vector útil para:**

- Enumeración de endpoints
- Descubrimiento de roles y lógica de privilegios
- Fuerza bruta o fuzzing de parámetros
- Pruebas de bypass de autenticación y autorización

Rutas funcionales internas del frontend detectadas

El archivo contiene funcionalidades administrativas del sistema relacionadas con usuarios:

- Crear usuario
- Editar usuario
- Eliminar usuario
- Asignar o revocar suscripciones
- Modificar roles en proyectos

Esto permite inferir que la aplicación tiene un **panel administrativo con funciones sensibles** y una API asociada.

---

2. Llamadas a funciones del backend (mutations)

Aparecen múltiples llamadas al objeto D, que parece ser un **servicio API centralizado**. Por ejemplo:

```
D.createUser({ requestBody: l })

D.updateUser({ userId: i.id, requestBody: t })

D.deleteUser({ userId: r })

D.assignSubscriptionToUser({ userId: r.userId, subscriptionId: r.subscriptionId })

D.getUserSubscriptionById({ userId: i })
```

Esto nos revela varios posibles endpoints backend REST o GraphQL:

```
POST /api/users              # Crear usuario

PUT  /api/users/{id}         # Actualizar usuario

DELETE /api/users/{id}       # Eliminar usuario

POST /api/users/{id}/subscription

GET  /api/users/{id}/subscription
```

"https://da[redacted]/assets/notifications-DBNLbAwn.js ",

Roles identificados

Desde el frontend se manejan tres tipos de rol:

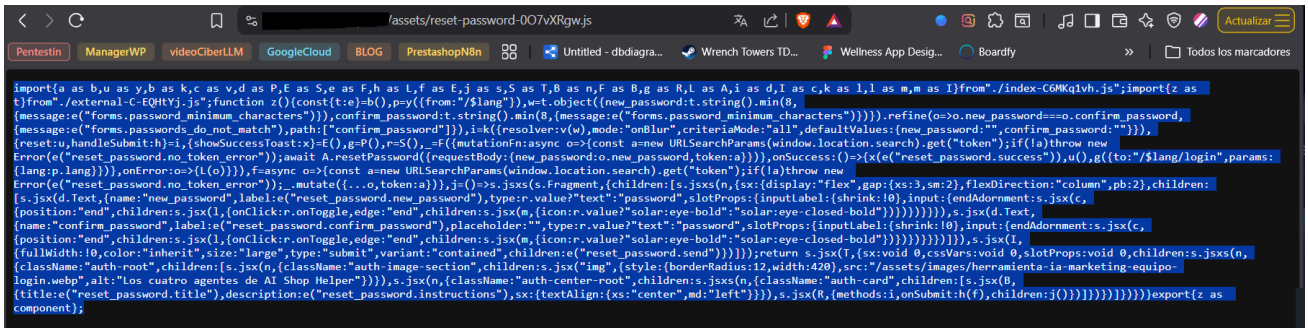
Administrador
Editor
Miembro

Y se observa la lógica para modificar estos roles:

```
F.updateUserProjectRole({

  projectId: a.id,

  requestBody: {

    is_administrator: true/false,

    is_editor: true/false

  }

})
```

Esto permite testear si un usuario sin privilegios puede modificar su propio rol o el de otro mediante manipulación de requests.

"https://da[redacted]/assets/reset-password-0O7vXRgw.js "



1. Token expuesto en URL (GET)

```
const a=new URLSearchParams(window.location.search).get("token");
```

El token de restablecimiento de contraseña se obtiene desde **URL** (?token=..). Esto tiene implicancias importantes:

El token podría quedar en el historial del navegador
Podría ser filtrado por referers si el usuario hace clic en enlaces externos.
Podría ser interceptado en caso de red comprometida si no hay HTTPS.
No se invalida automáticamente después de uso (no se ve en el frontend al menos)

Este uso de tokens en URL sin medidas de seguridad adicionales es una **muy buena superficie para ataques**. Si el token no tiene tiempo de expiración o no se invalida correctamente, puede ser reutilizado por un atacante.

"https://da[redacted]/assets/external-C-EQHtYj.js" HACER PRUEBAS DE PENTEST "pruebas fuertes"

Este módulo es ALTAMENTE explotable si el backend no es perfecto. aqui se confirma la utilización de **Reactjs** y no solo eso si no que hay una superficie enorme de ataque

Validaciones de formularios hechas con **Zod**

Manejo de formularios react-hook-form

Manejo de datos (fetch/mutaciones) react-query

Los vectores más fuertes:

1. Escalada a superusuario
2. Toma de cuentas
3. Escalada de privilegios en proyectos
4. Fraude de créditos
5. CSRF destructivo
6. Enumeración de usuarios

"https://da[redacted]/assets/index-lLGzNfvJ.js"

No es una fuente de vulnerabilidades directa, pero este componente **sirve como punto de observación**. Si se logra explotar algo en el backend (creación de usuario, XSS, modificación de tokens), este tipo de componente puede reflejar los efectos.

"https://da[redacted]/assets/container-BCnAXgx-.js"

Este componente **no tiene lógica sensible, ni acceso a datos, ni manipula el DOM activamente**. Sin embargo:

Naming + export `export { n as C }`

Sabemos que este es el componente `C` que usan como `<C>...</C>` en otros lugares, como en el archivo anterior (`index-lLGzNfvJ.js`)

2.Rutas API Identificadas para analizar posteriormente

Rutas API identificadas:

- /api/auth/login - Autenticación de usuarios
- /api/auth/register - Registro de usuarios
- /api/auth/reset-password - Reset de contraseñas
- /api/products - Gestión de productos
- /api/subscriptions - Suscripciones
- /api/billing - Facturación
- /api/appearance - Configuración de apariencia
- /api/calendar - Funcionalidad de calendario
- /api/questions - Sistema de preguntas
- /api/pagination - Paginación
- /api/projects - Gestión de proyectos
- /api/account - Gestión de cuentas
- /api/admin - Panel administrativo

3.ARCHIVOS DE DESCARGA ENCONTRADO “POSIBLE HONEYPOT” O DISTRACCION POR CLOUDFLARE.

Nota de seguridad

Aunque se han identificado y localizado los archivos mencionados, se ha decidido aplicar **un flujo de seguridad controlado** antes de cualquier interacción directa con los mismos.

Existen **múltiples indicadores sobre todo lo generico del encabezado** de **zado.cloud.com** que sugieren que dichos archivos podrían haber sido **deliberadamente expuestos con fines de detección u ocultos tras la dirección** actuando como **artefactos señuelo (honeypot / canary files)** diseñados para atraer a potenciales atacantes.

Este tipo de archivos puede incorporar **mecanismos de balizaje** tales como solicitudes externas, fingerprinting del entorno o técnicas de correlación de eventos, orientadas a **identificar, rastrear o perfilar la dirección IP y el comportamiento del actor que accede a ellos**.

Por este motivo, cualquier análisis posterior se realizará exclusivamente dentro de **entornos aislados y controlados**, siguiendo procedimientos de seguridad que minimicen la exposición, eviten la atribución directa y permitan evaluar el riesgo sin comprometer la infraestructura ni la identidad del analista.

Resumen de flujo seguro:

1. Activa VPN verifica IP.
2. Descarga con wget script.
3. Analiza con Scrip para hacer fuzzin de posibles paht.
4. Si es seguro, abre solo en entorno aislado.
5. Nunca se permiten3_ ejecución automática ni macros.

"https://www.cloud.com/s/[redacted]/xl_david-blaine_component_tanzania_books.pdf"

"https://www.cloud.com/s[redacted]6/socio_respectively_366996.pptx"

"https://www.cloud.com/s[redacted]6/socio_ahead_531437_sweden_popup.wav"

"https://www.cloud.com/s[redacted]6/money-popup-crack.pdf"

"https://www.cloud.com/s[redacted]6/indonesia-quito-nancy-grace-left-glad.xlsx"

"https://www.cloud.com/s[redacted]6/nauru-6015-small-fighter-left-gender.psd"

"https://www.cloud.com/s[redacted]6/large_news.txt"

"https://www.cloud.com/s[redacted]6/vintage_bahrain_saipan.xls"

"https://www.cloud.com/s[redacted]6/footer-243-ecuador.iso"

"https://www.cloud.com/s[redacted]6/large_energy_dry_philippines.rar"

"https://www.cloud.com/s[redacted]6/gustavia-entertainment-productivity.docx"

"https://www.cloud.com/s[redacted]6/footer-barbados-celine-dion.ppt"

"https://www.cloud.com/s[redacted]6/design_suriname_2015.mp3"

"https://www.cloud.com/s[redacted]6/expertise_2015_conakry_sao-tome-and-principe_gender.mp4"

"https://www.cloud.com/s[redacted]6/india-data-large-gk-chesterton-mother.eps"

"https://www.cloud.com/s[redacted]6/tv-xs.doc"

"https://www.cloud.com/s[redacted]6/trinidad_samuel-morse_bring.m4v"

"https://www.cloud.com/s[redacted]6/legislation-grain.zip"

"https://www.cloud.com/s[redacted]6/kyrgyzstan-04795009-picabo-street-guide-style.ai"

)

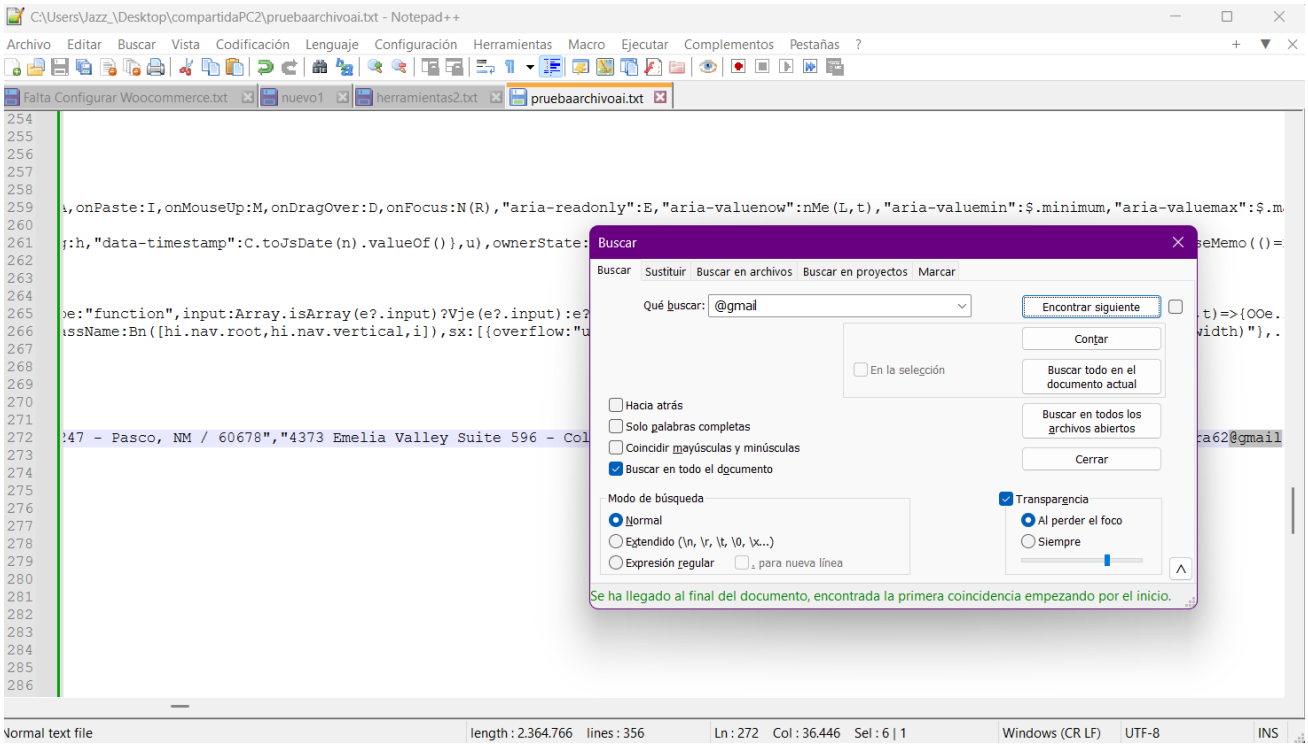
4.Extraccion Email para analizar posteriormente.

Durante la fase de reconocimiento se han identificado varias **direcciones de correo electrónico potencialmente asociadas al objetivo**. No obstante, antes de realizar cualquier interacción directa, se ha establecido un **enfoque OSINT estricto y pasivo** para su análisis.

Existen **indicadores razonables** de que dichas direcciones podrían haber sido **expuestas de forma intencionada**, actuando como **señuelos** con el objetivo de detectar actividades de enumeración, recolección o contacto no autorizado. En determinados escenarios, este tipo de cuentas puede estar configurado para **registrar intentos de validación, recepción de mensajes, correlación temporal o análisis de origen**, permitiendo inferir información sobre el actor que las investiga.

Por este motivo, el análisis se limitará inicialmente a **técnicas OSINT pasivas**, incluyendo correlación en fuentes abiertas, brechas públicas conocidas, patrones de reutilización, metadatos históricos y presencia en servicios de terceros **sin generar tráfico directo ni interacción activa** con los sistemas asociados.

El objetivo de este enfoque es **determinar si las direcciones corresponden a identidades reales, cuentas operativas, artefactos de detección o simples cebos**, así como evaluar si pueden aportar información útil adicional sin comprometer la trazabilidad ni la seguridad del proceso de análisis.



1. Gmail.com (9 correos)

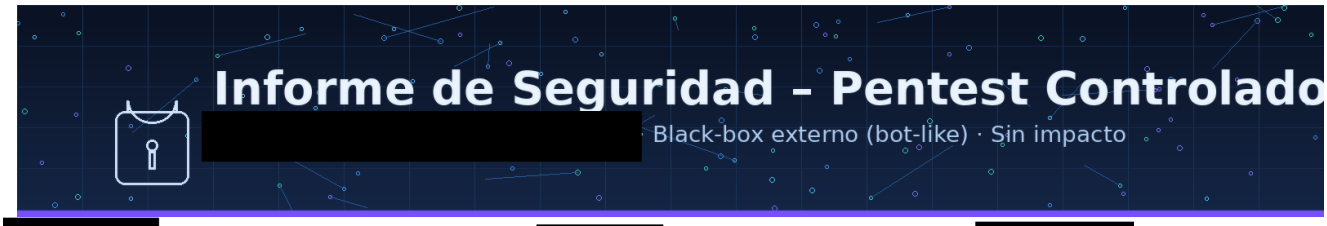
[redacted]der31@gmail.com
[redacted]s@gmail.com
[redacted]39@gmail.com
[redacted]94@gmail.com
[redacted]ara62@gmail.com

2. Hotmail.com (9 correos)

[redacted]evieve.powlowski@hotmail.com
[redacted]um27@hotmail.com
[redacted]s@hotmail.com
[redacted]@hotmail.com
[redacted]@hotmail.com

3. Yahoo.com (6 correos)

[redacted] violet.ratke86@yahoo.com
[redacted]hickle4@yahoo.com
[redacted] benny89@yahoo.com
[redacted].block85@yahoo.com
[redacted]nie.abernathy70@yahoo.com



[redacted]4@gmail.com [redacted]2@hotmail.com [redacted]owitz24@yahoo.com

[redacted]@gmail.com [redacted]vito.hudson@hotmail.com

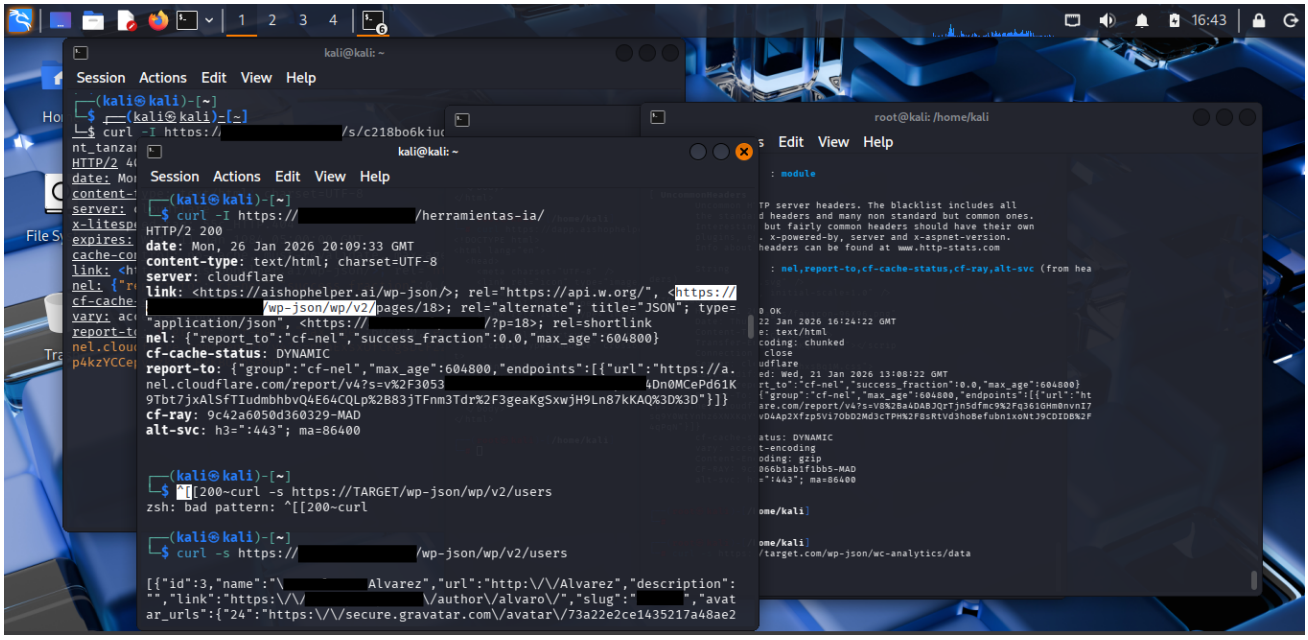
[redacted]@gmail.com [redacted]e.gerhold73@hotmail.com

[redacted]@gmail.com [redacted] avery43@hotmail.com

5.Endpoint en https:[redacted].ai/herramientas-ia/

Estos endpoints pueden ser usados para obtener información sobre usuarios, configuraciones, plugins, contenido, etc.

Algunos endpoints pueden estar mal configurados y permitir acceso sin autenticación (ej: /wp/v2/users puede filtrar nombres de usuario si no está restringido).



https://[redacted]/wp-json/wp/v2/users

https://[redacted]i/wp-json/wp/v2/posts

https://[redacted]i/wp-json/wp/v2/pages

https://[redacted]i/wp-json/wp/v2/comments

https://[redacted]i/wp-json/wp/v2/media

https://[redacted]i/wp-json/contact-form-7/v1/

https://[redacted]i/wp-json/jetpack/v4/

https://[redacted]i/wp-json/yoast-api/v1/

https://[redacted]/wp-json/woocommerce/v3/

https://[redacted]/wp-json/mailchimp/v1/

https://[redacted]/wp-json/elementor/v1/

https://[redacted]/wp-json/wc-paypal/v1/

6. Plugins Detectados.

WPSCAN:PRUEBA

Posibles vectores de ataque

Basado en los plugins y rutas:

- **Contact Form 7**: Si hay un formulario accesible, podría ser vulnerable a XSS o spam si no hay validaciones.
- **WooCommerce**: Exponer `/wc/v3/` puede permitir explorar productos, clientes o pedidos si hay configuraciones erróneas.
- **Jetpack**: Tiene múltiples endpoints con funcionalidades administrativas que podrían ser abusadas si las claves de autenticación no están protegidas.
  - Se observan configuraciones como `jetpack_waf_automatic_rules` o `jetpack_protect_key`, pero no se especifica si están habilitadas.
  - No se observa uso claro de autenticación fuerte (`jetpack_sso_require_two_step` aparece como `false` por defecto).

El archivo revela múltiples plugins activos:

- Jetpack
- Akismet
- Contact Form 7
- Elementor
- WooCommerce
- Mailchimp
- Yoast SEO
- Pinterest integration
- PayPal

7.WPSCAN PRUEBAS POC.

```
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   __  __ _ _ __ ®
           \ \/  \/ / |  ___/ \___ \ / _|/ _` | '_ \
            \  /\  /  | |     ____) | (_| (_| | | | |
             \/  \/   |_|    |_____/ \__|\__,_|_| |_|

         WordPress Security Scanner by the WPScan Team
                         Version 3.8.28
      Sponsored by Automattic - https://automattic.com/
      @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[+] URL: https:[redacted]/ [188.114.96.5]
[+] Started: Wed Jan 28 07:02:45 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entries:
 |  - server: cloudflare
 |  - report-to: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/r
eport/v4?s=vWNmALiFGAz2[redacted]15NOJTm5H7eP9kAfzlzr
6K%2FiETNsrgMRgCTQ%3D%3D"}]}
 |  - nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
 |  - cf-cache-status: DYNAMIC
 |  - cf-ray: 9c505780df29f767-MAD
 |  - alt-svc: h3=":443"; ma=86400
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] robots.txt found: https://[redacted]/robots.txt
 | Found By: Robots Txt (Aggressive Detection)
 | Confidence: 100%

[+] XML-RPC seems to be enabled: https://[redacted]/xmlrpc.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
 | References:
```

Hallazgos relevantes y brechas de seguridad:

1. Exposición del archivo readme.html

Ruta *: https://[redacted]i/readme.html*
Esto confirma que el sitio está utilizando WordPress y permite inferir la versión fácilmente. Este archivo no debería estar accesible públicamente.

---

2. XML-RPC habilitado

Ruta *: https://[redacted]i/xmlrpc.php*
La presencia de XML-RPC puede ser utilizada para:
Ataques de fuerza bruta por XML-RPC multicall.
Escaneos tipo pingback (para DDoS distribuido).
Enumeración de usuarios.
Este vector sigue siendo ampliamente explotado.

3. wp-cron.php expuesto

Ruta : https://[redacted]i/wp-cron.php
Puede ser utilizado para generar carga innecesaria en el servidor (DDoS lento).
El endpoint debería estar protegido con validación o restringido.

---

4. Enumeración de usuarios

Usuarios identificados:
jose, alvaro, daniel, helper, paola, enrique
Vías:
https://[redacted]i/wp-json/wp/v2/users/
https://[redacted]i/author-sitemap.xml
Patrón de autores y brute forcing.
La exposición de usuarios permite ataques de fuerza bruta y phishing dirigidos.

```
┌──(kali㉿kali)-[~]
└─$ curl -s https://[redacted]/wp-json/wp/v2/users

[{"id":3,"name":"\[redacted] Alvarez","url":"http:\/\/[redacted]","description":
"","link":"https:\/\/[redacted]\/author\/alvaro\/","slug":"alvaro","avat
ar_urls":{"24":"https:\/\/secure.gravatar.com\/avatar\/73a22e2ce1435217a48ae2
f7c[redacted]?s=24&d=mm&r=g","48":"https:\/\/sec
ure.gravatar.com\/avatar\/73a22[redacted]
4e8e08c05dd7e?s=48&d=mm&r=g","96":"https:\/\/secure.gravatar.com\/avatar\/73a
22e2ce14352[redacted]=96&d=mm&r=g"}
,"meta":{"jetpack_donation_warning_dismissed":false},"pll_duplicate_content":
false,"yoast_head":"<!— This site is optimized with the Yoast SEO plugin v26
.8 - https:\/\/yoast.com\/product\/yoast-seo-wordpress\/ —>\n<title>\[redacted]
aro [redacted], autor en [redacted]<\/title>\n<meta name=\"robots\" content=
\"index, follow, max-snippet:-1, max-image-preview:large, max-video-preview:-
1\" \/>\n<link rel=\"canonical\" href=\"https:\/\/[redacted]\/author\/al
varo\/\" \/>\n<meta property=\"og:locale\" content=\"es_ES\" \/>\n<meta prope
rty=\"og:type\" content=\"profile\" \/>\n<meta property=\"og:title\" content=
```

---

5. Plugins desactualizados

Elementor
Versión detectada: 3.34.2
Última versión disponible: 3.34.3
Si esa versión intermedia corrige vulnerabilidades, puede haber una brecha aprovechable.

YITH WooCommerce Wishlist
Versión detectada: 4.11.0
Última versión: 4.12.0
Este tipo de plugins comerciales suelen tener histórico de vulnerabilidades de XSS o CSRF.

---

6. Temas desactualizados

TwentyTwentyTwo

Versión instalada: *2.0*
Última versión: *2.1*
Aun sin ser el tema activo, es una superficie de ataque.

---

7. WordPress REST API expuesta

Ruta accesible: *https://[redacted]i/wp-json/wp/v2/users/*
Esto habilita:
Enumeración de usuarios.
Posibles enumeraciones de contenido, taxonomías, endpoints personalizados mal implementados.

---

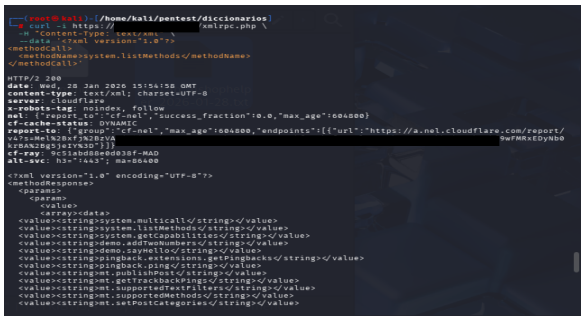
8. Plugin sin versión detectada: ***quiety-core***

Este plugin probablemente está vinculado al tema principal y puede contener funcionalidades personalizadas. Si no está actualizado o auditado, representa un vector de ataque poco visible.

Confirmación que el endpoint ***https://ai[redacted]rpc.php *** ***está habilitado, funcional y***
expone una gran superficie de ataque ***.***

Prueba: el servidor responde con con los metodos los cuales algunos son peligrosos

```
curl -i https://ais[redacted]c.php \
  -H "Content-Type: text/xml" \
  --data '<?xml version="1.0"?>
<methodCall>
  <methodName>system.listMethods</methodName>
</methodCall>'
```



Lo crítico: Métodos disponibles

Entre los métodos devueltos, se destacan varios altamente sensibles desde el punto de vista ofensivo:

1. system.multicall

Permite enviar múltiples métodos en una sola solicitud. Es el ***método más explotado*** *para:*

Ataques de fuerza bruta por diccionario.
Ampliar el volumen del ataque sin ser bloqueado.

Con un solo request se pueden probar cientos de combinaciones usuario/clave sin levantar alertas como con un ataque clásico.

---

2. pingback.ping y pingback.extensions.getPingbacks

Se utilizan para ***DDoS distribuido*** *desde servidores WordPress legítimos contra víctimas externas.*
También se usan para escanear redes internas si el servidor es vulnerable.

---

3. ***metaWeblog.*** , ***blogger.*** , ***wp.****

Exponen métodos de:

Lectura, edición y publicación de posts (*wp.newPost*, *metaWeblog.newPost*, *wp.uploadFile*, *wp.getUsers*, *wp.getProfile*, ***Manejo de medios, usuarios, comentarios y términos*** *wp.editProfile*).
Manipulación de categorías/taxonomías .
Si un atacante consigue credenciales válidas, puede ***tomar control total del sitio*** *mediante XML-RPC sin necesidad de entrar al panel web.*

Recomendaciones:

Deshabilitar XML-RPC *si no se usa (por ejemplo, con plugins como "Disable XML-RPC").*
Deshabilitar completamente el archivo ***xmlrpc.php*** *si no se utiliza, agregando esta regla en* ***Eliminar archivos innecesarios como*** ***readme.html*** *y restringir acceso a* *.htaccess* *:*
Actualizar Elementor y YITH Wishlist *wp-cron.php*.
Revisar el plugin ***quietycount*** *cuanto antes para verificar si está actualizado y libre de* ***Ocultar REST API pública o aplicar autenticación*** *vulnerabilidades.* *para utilizadores sensíveis como* ***Aplicar seguridad a nivel WAF / Cloudflare Rules*** *para bloquear escaneos automáticos y bots.* *wp/v2/users*

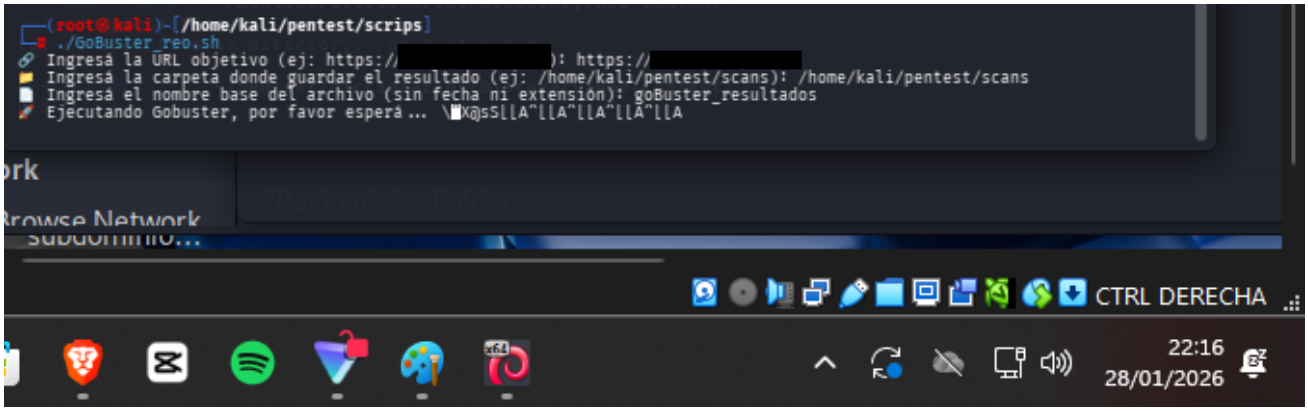
Gobuster:

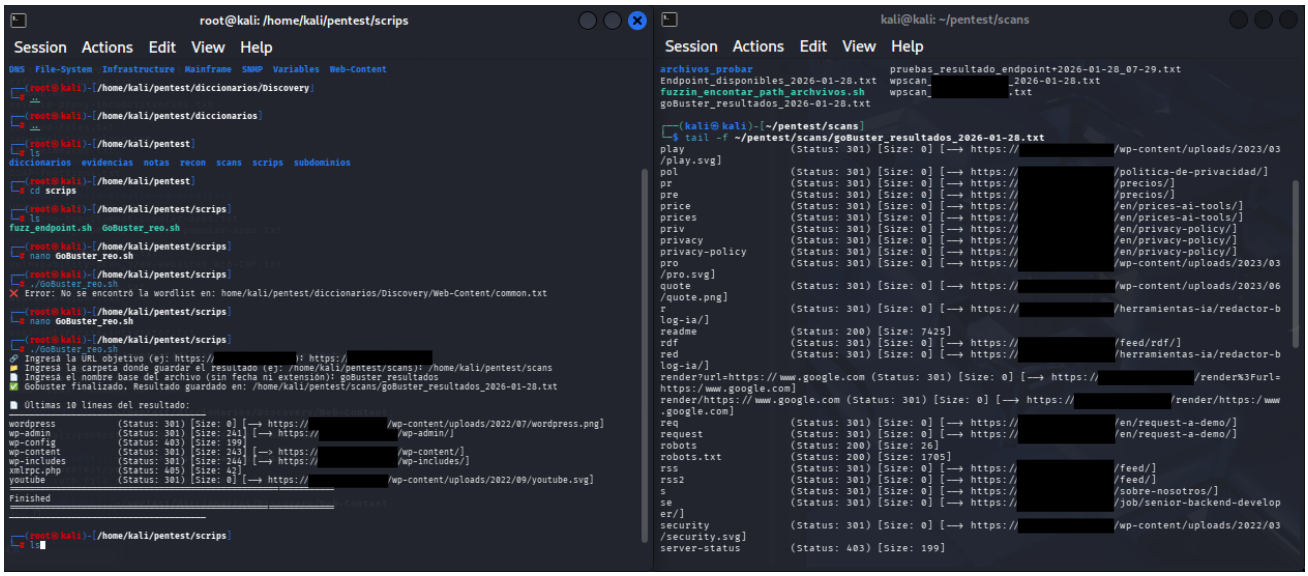
Se crea un pequeño scrip para automatizar el guardado y escaneo ya que tenemos varias direcciones a las cuales se les pasara gobuster.

Hora : 22:18 para no ralentizar servidores y para ser prudente

(sitio en producción y con Cloudflare)

1. **No se usa más de 10 threads** ya que no se la capacidad del backend.
2. Si Cloudflare detecta el escaneo, podre ver errores tipo 403, 429 o bloqueos temporales.
3. E agregado un user-agent para simular tráfico legítimo: (-a "Mozilla/5.0 (X11; Linux x86_64)")



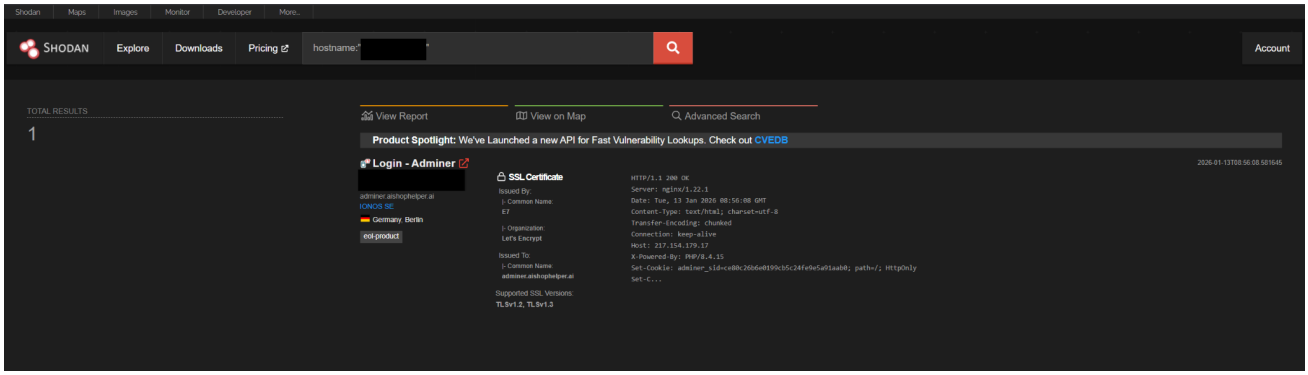


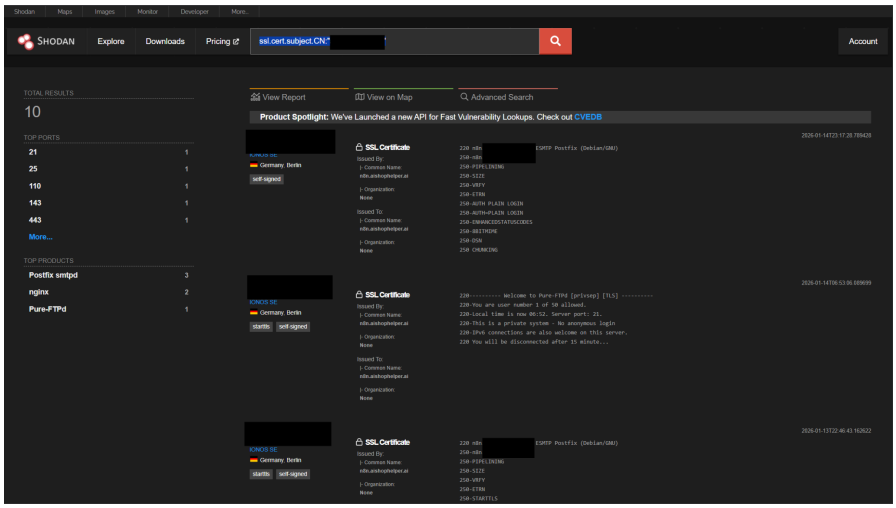
SHODAN:

Exposición de servicio administrativo detectada en fuentes OSINT (Shodan).

Se identifica el subdominio adminer[redacted] accesible públicamente, resolviendo a una IP fuera de Cloudflare y exponiendo el panel Adminer sobre servidor NGINX + PHP, lo que incrementa la superficie de ataque. Este hallazgo confirma la existencia de un servicio sensible accesible desde

Internet y susceptible de pruebas adicionales de autenticación, hardening y control de acceso.Aunque mayormente con tan pocos puertos abiertos muy posible honeypot y ubuntu virtualizado





6. Hallazgos (tabla) y priorización

6.1 Criterio de severidad (simple)

Severidad = Impacto (Bajo/Medio/Alto) × Probabilidad (Baja/Media/Alta).

	Prob. Baja	Prob. Media	Prob. Alta
Impacto Bajo	Bajo	Bajo	Medio
Impacto Medio	Bajo	Medio	Alto
Impacto Alto	Medio	Alto	Crítico

6.2 Tabla de hallazgos (rellenar)

ID	Severidad	Título	Descripción	Evidencia (req/resp/captura)	Riesgo/impacto	Recomendación (quick win)
001	MEDIA/ALTA	NGINX 1.22.1	Grieta por CVE y version antigua		MEDIO	Actualizar a ultima versión estable 1.28.0

7. Recomendaciones (quick wins) y plan de mitigación

7.1 Quick wins (aplicables en 24–72h)

Activar backups + prueba de restauración (prioridad absoluta).
Restringir acceso al origen (VPS) a rangos de IP de Cloudflare para 443/80.
Asegurar webhooks n8n con secreto/firma + rate limiting + validación de origen.
Ajustar headers de seguridad (HSTS, CSP, Frame/Content-Type/Referrer/Permissions).
Revisar cookies (Secure/HttpOnly/SameSite) y tiempos de sesión + logout invalida server-side.

7.2 Mitigación por prioridad (rellenar según hallazgos)

Prioridad	Acción	Owner	Fecha objetivo

8. Anexos

A. Endpoints descubiertos (pegar export aquí)

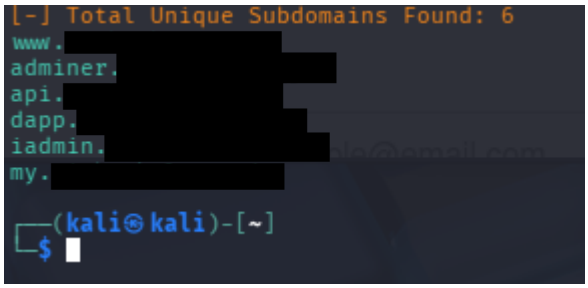
B. Capturas y evidencias (referencias a archivos).

SUBDOMINIOS:

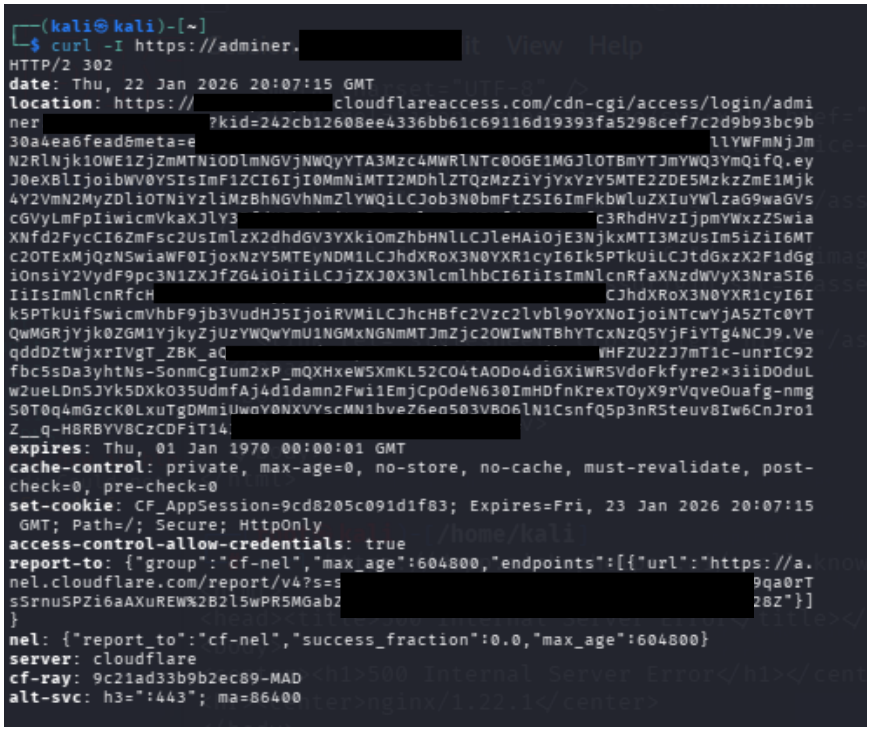
Herramientas: Sublister ,

Consultar de Certificados:

curl -s "https://crt.sh/?q=%[redacted]ai&output=json" | jq -r '.[].name_value' | sort -u



-Se hicieron pruebas de consulta más que de escaneo en general a todos los subdominios ,encontrando respuesta en 2 de ellos ,encontrando carpetas bien protegidas detrás de cloudflare pidiendo credenciales.



C. Configuración y notas (Cloudflare, WAF, firewall, composición de servicios).

Sugerencias de pruebas ofensivas (PoC recomendadas)

Escalada de privilegios por manipulación de updateUserProjectRole

Objetivo: Verificar si un usuario sin privilegios puede autoasignarse como administrado

```
POST /api/updateUserProjectRole

Content-Type: application/json

Authorization: Bearer [TOKEN]

{

  "is_administrator": true,

  "is_editor": false,

  "userProjectId": "[ID_del_usuario_actual_en_proyecto]"

}
```

Resultado esperado (si es vulnerable): el usuario se convierte en administrador.

Pruebas de concepto recomendadas

https://[redacted]ai/assets/admin-DRrG8g0t.js

Crear superusuario desde cuenta común

```
POST /api/users

Authorization: Bearer [token_limited]

Content-Type: application/json


{

  "email": "ataque@redteam.com",

  "full_name": "Red Team",
```

"password": "Password123!",

"confirm_password": "Password123!",

"is_superuser": true,

"is_active": true

}

Escalada de privilegios dentro de proyecto

POST /api/updateUserProjectRole

Authorization: Bearer [token_limited]

Content-Type: application/json

{

"is_administrator": true,

"is_editor": false,

"userProjectId": "[otro_usuario]"

}

Cambio de contraseña a otro usuario

PUT /api/users/[id]

Authorization: Bearer [token_limited]

Content-Type: application/json

{

"password": "Pwned123!",

"confirm_password": "Pwned123!"

}

Modificación de créditos de proyecto

POST /api/updateProjectAssociatedTokens

Authorization: Bearer [token_limited]

Content-Type: application/json

```
{
  "projectId": "123",
  "associated_tokens": 99999999
}
```

Conclusión

Este módulo representa una **zona de alto impacto** desde la perspectiva ofensiva. Ofrece múltiples puntos críticos que deben ser reforzados desde el backend, ya que el frontend muestra:

- Falta de control de privilegios en funcionalidades críticas.
- Capacidad de modificar o eliminar usuarios, contraseñas, proyectos y créditos.
- Ausencia de mecanismos de auditoría, verificación en dos pasos o protección contra automatización.

Pruebas de concepto recomendadas

https://[redacted]ai/assets/external-C-EQHtYj.js

RED TEAM – CHECKLIST DE PRUEBAS (basado en el código)

ID	Categoría	Prueba	Objetivo	Método	Requiere Auth	Riesgo
P1	Escalada de privilegios	Crear usuario con is_superuser: true	Tomar control admin	POST /api/users con campos forzados	Sí	Crítico
P2	Toma de cuentas	Editar otro usuario y cambiarle la contraseña	Acceso persistente / lateral movement	PUT /api/users/:id	Sí	Crítico
P3	Autopromoción en proyecto	Ascenderse a admin de proyecto propio	Acceso a funciones restringidas	POST /api/updateUserProjectRole	Sí	Alto
P4	Asignación masiva de créditos	Cambiar créditos asociados a proyecto	Fraude lógico / DoS	POST /api/updateProjectAssociatedTokens	Sí	Alto
P5	Eliminar proyectos ajenos	Usar deleteProject() sin ser admin	Destrucción de datos	DELETE /api/projects/:id	Sí	Crítico si no hay checks
P6	CSRF	Ejecutar borrado desde HTML malicioso	Acceso sin interacción directa	HTML+JS con cookies del usuario	No si hay CSRF token	Crítico
P7	Enumeración de usuarios	Listar usuarios con paginación y búsqueda	Recopilar emails válidos	GET /api/users?search=	Sí (o testing sin auth)	Medio

P8	**Cambio de suscripción**	**Cambiar plan de otro usuario**	**Fraude de privilegios**	**POST /api/assignSubscriptionToUser**	**Sí**	**Alto**
P9	**Validación de password frontend-only**	**Enviar password corta o inválida**	**Validar si el backend filtra**	**POST /api/users con "password": "123"**	**Sí**	**Medio**